The proto file for contact & invoice:

```
syntax = "proto3";
import "google/protobuf/duration.proto";
import "google/protobuf/timestamp.proto";
import "google/protobuf/wrappers.proto";

option csharp_namespace = "GrpcDemo";

service Invoice {
  rpc CreateAddress(CreateAddressRequest) returns (ActionReply);
  rpc AddInvoiceItem(AddInvoiceItemRequest) returns (ActionReply);
  rpc UpdateInvoiceDueDate(UpdateInvoiceDueDateRequest) returns (ActionReply);
}

service Contact {
  rpc CreateContact(CreateContactRequest) returns (CreateContactResponse);
}

message ActionReply {
  bool success = 1;
  string message = 2;
}

enum InvoiceStatus {
  INVOICE_STATUS_UNKNOWN = 0;
  INVOICE_STATUS_DRAFT = 1;
  INVOICESTATUS_AWAIT_PAYMENT = 2;
  INVOICE_STATUS_PAID = 3;
  INVOICE_STATUS_OVERDUE = 4;
}

message CreateAddressRequest {
  string street = 1;
  string city = 2;
  string state = 3;
  string zip_code = 4;
  string country = 5;
}

message CreateContactRequest {
  string first_name = 1;
  string last_name = 2;
  string email = 3;
  string phone = 4;
  int32 year_of_birth = 5;
  bool is_active = 6;
}

message CreateContactResponse {
  string contact_id = 1;
}

message UpdateBatchInvoicesStatusRequest {
      repeated string invoice_ids = 1;
      InvoiceStatus status = 2;
}
```

```
message UpdateInvoicesStatusRequest {
  map<string, InvoiceStatus> invoice_status_map = 1;
}

message AddInvoiceItemRequest {
  string name = 1;
  string description = 2;
  google.protobuf.DoubleValue unit_price = 3;
  google.protobuf.Int32Value quantity = 4;
  google.protobuf.BoolValue is_taxable = 5;
}

message UpdateInvoiceDueDateRequest {
  string invoice_id = 1;
  google.protobuf.Timestamp due_date = 2;
  google.protobuf.Duration grace_period = 3;
}
```

****

To safely remove a field, we must ensure that the removed field number is not being used in th future. To avoid any potential conflicts, we can reserve the removed field number by using th `reserved` keyword. For example, if we delete the `year_of_birth` and `is_active` fields fro the `CreateContactRequest` message, we can reserve the field numbers, as follows:

```
message CreateContactRequest {
  reserved 5, 6;
  reserved "year_of_birth", "is_active";
```



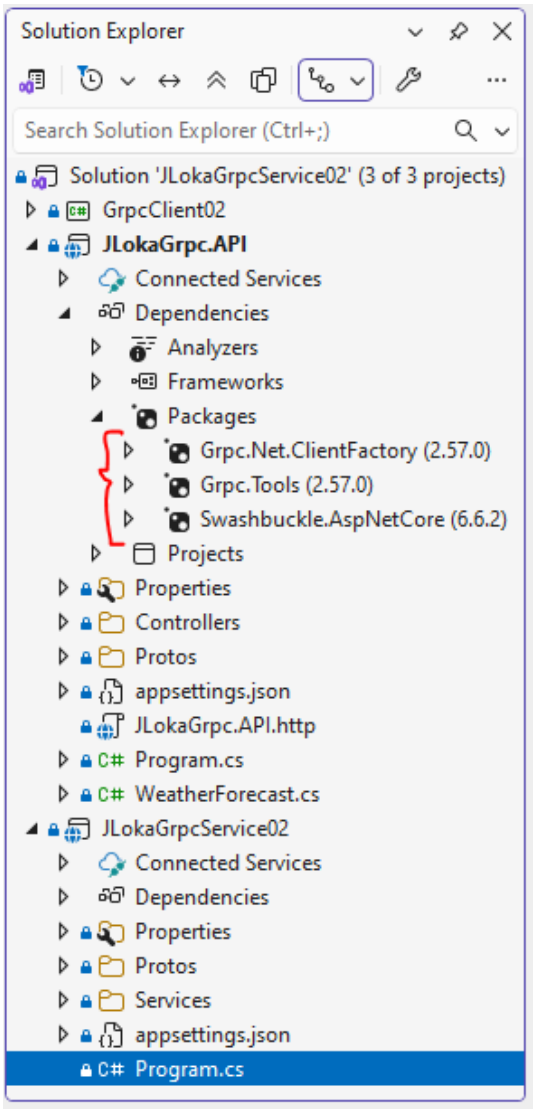
```
  string first_name = 1;
  string last_name = 2;
  string email = 3;
  string phone = 4;
}
```

****

If we try to use a reserved field number or field name, the gRPC tooling will report an error. Note that the reserved field names should be listed, as well as the reserved field numbers. This ensures that the JSON and text formats are backward compatible. When the field names are reserved, they cannot be placed in the same reserved statement with the field numbers.

****

To use ASP.Net Core API with gRPC Service we need to install:



****

Then to configure Web API to use the client:

```
builder.Services.AddGrpcClient<Contact.ContactClient>(x => x.Address = new
Uri("https://localhost:7214"));
```

****

And to use it with controller:

```
using GrpcDemo;
using Microsoft.AspNetCore.Mvc;

namespace JLokaGrpc.API.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class ContactController (Contact.ContactClient client,
ILogger<ContactController> logger) : ControllerBase
    {
        [HttpPost]
        public async Task<IActionResult> CreateContact(CreateContactRequest request)
        {
            var replay = await client.CreateContactAsync(request);
            logger.LogInformation($"Replay is: {replay.ContactId}");
            return Ok(replay);
        }
    }
}
```

****

Mutations are used to modify data in GraphQL. A mutation consists of three parts:

- **Input**: The input is the data that will be used to modify the data. It is named with the `Input` suffix following the convention, such as `AddTeacherInput`.
- **Payload**: The payload is the data that will be returned after the mutation is executed. It is named with the `Payload` suffix following the convention, such as `AddTeacherPayload`.
- **Mutation**: The mutation is the operation that will be executed. It is named as *verb + noun* following the convention, such as `AddTeacherAsync`.

****

Defining a GraphQL schema

Usually, a system has multiple types of data. For example, a school management system has teachers, students, departments, and courses. A department has multiple courses, and a course has multiple students. A teacher can teach multiple courses, and a course can be taught by multiple teachers as well. In this section, we will discuss how to define a GraphQL schema with multiple types of data.

Scalar types

Scalar types are the primitive types in GraphQL. The following table lists the scalar types in GraphQL:

Scalar type	Description	.NET type
`Int`	Signed 32-bit integer	`int`
`Float`	Signed double-precision floating-point value specified in IEEE 754	`float` or `double`
`String`	UTF-8 character sequence	`string`
`Boolean`	`true` or `false`	`bool`
`ID`	A unique identifier, serialized as a string	`string`

****

Besides the preceding scalar types, `HotChocolate` also supports the following scalar types:

- `Byte`: Unsigned 8-bit integer
- `ByteArray`: Byte array that is encoded as a Base64 string
- `Short`: Signed 16-bit integer
- `Long`: Signed 64-bit integer
- `Decimal`: Signed decimal value
- `Date`: ISO-8601 date
- `TimeSpan`: ISO-8601 time duration
- `DateTime`: A custom GraphQL scalar defined by the community at `https://www.graphql-scalars.com/`. It is based on RFC3339. Note that this `DateTime` scalar uses an offset to UTC instead of a time zone
- `Url`: URL
- `Uuid`: GUID
- `Any`: A special type that is used to represent any literal or output type

****

GraphQL supports enumerations as well. Enumeration types in GraphQL are a special kind of scalar type. They are used to represent a fixed set of values. .NET supports enumeratiopn types very well so that you can use the .NET enum type directly in GraphQL. You can define an enumeration type as follows:

```
public enum CourseType
{
    Core,
    Elective,
    Lab
}
```

The preceding code defines an enumeration type named `CourseType` with three values: `Core`, `Elective`, and `Lab`. The generated GraphQL schema is as follows:

```
enum CourseType {
  CORE
  ELECTIVE
  LAB
}
```

****

The preceding code defines a `Teacher` type and a `Department` type. The `Teacher` type has a `Department` property of the `Department` type. HotChocolate will generate the schema as follows:

```
type Teacher {
  id: UUID!
  firstName: String!
  lastName: String!
  departmentId: UUID!
  department: Department!
}

type Department {
  id: UUID!
  name: String!
  description: String
}
```

****

```
public class Department
{
    public Guid Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }
    public List<Teacher> Teachers { get; set; } = new();
}
```

The generated schema is as follows:

```
type Department {
  id: UUID!
  name: String!
  description: String
  teachers: [Teacher!]!
}
```

****

Every GraphQL service must have a Query type, but may or may not have a Mutation type. So the teachers query is actually a field of the Query type, just like the department field in the teacher type. Mutations work in the same way.

****

```
type Query {
  teachers: [Teacher!]!
  teacher(id: UUID!): Teacher
}

type Mutation {
  addTeacher(input: AddTeacherInput!): AddTeacherPayload!
}
```

****

The query type for getting teacher or teachers:

```
using JLokaTestGraphQL.Data;
using JLokaTestGraphQL.Models;
using Microsoft.EntityFrameworkCore;

namespace JLokaTestGraphQL.GraphQL.Types;
public class Query
{
    public async Task<List<Teacher>> GetTeachers([Service] AppDbContext context) =>
await context.Teachers.Include(x => x.Department).ToListAsync();

    public async Task<Teacher?> GetTeacher(Guid id, [Service] AppDbContext context)
=> await context.Teachers.FindAsync(id);
}
```

The mutation for add teacher is:

```
public record AddTeacherInput(string FirstName, string LastName, string Email,
string? Phone, string? Bio, Guid departmentId);
```

```
using JLokaTestGraphQL.Models;

namespace JLokaTestGraphQL.GraphQL.Mutations;
public class AddTeacherPayload
{
    public Teacher Teacher { get; }
    public AddTeacherPayload(Teacher teacher)
    {
        Teacher = teacher;
    }
}
```

```
using JLokaTestGraphQL.Data;
using JLokaTestGraphQL.Models;

namespace JLokaTestGraphQL.GraphQL.Mutations;
public class Mutation
{
public async Task<AddTeacherPayload> AddTeacherAsync(AddTeacherInput input,
[Service] AppDbContext context)
    {
        var teacher = new Teacher
        {
            Id = Guid.NewGuid(),
            FirstName = input.FirstName,
            LastName = input.LastName,
            Email = input.Email,
            Phone = input.Phone,
            Bio = input.Bio,
            DepartmentId = input.departmentId,
        };

        context.Teachers.Add(teacher);
        await context.SaveChangesAsync();
        return new AddTeacherPayload(teacher);
    }
}
```

****

A resolver is a function that is used to retrieve data from somewhere for a specific field. The resolver is executed when the field is requested in the query. The resolver can fetch data from a database, a web API, or any other data source. It will drill down the graph to retrieve the data for the field. For example, when the department field is requested in the teachers query, the resolver will retrieve the Department object from the database. But when the query does not specify the department field, the resolver will not be executed. This can avoid unnecessary database queries.

****

HotChocolate supports three ways to define schemas:

• **Annotation-based**: The first way is to use the annotation-based approach, which is what we have been using so far. **HotChocolate** automatically converts public properties and methods to a resolver that retrieves data from the data source following conventions. If a method has a **Get prefix** or **an Async suffix**, these prefixes or suffixes will be removed from the name.

• **Code-first**: This approach allows you to define the schema **using explicit types and resolvers**. It uses the Fluent API to define the details of the schema. This approach is more flexible when you need to customize the schema.

• **Schema-first**: This approach **allows you to define the schema** using the GraphQL schema definition language.

****